Face detection updated code (Brittleness Check)

AUTHOR
Shreya Shetty

Shreya's Final R Code for DSPP Week 2 Presentation on Face Detection (Brittleness by Variable)

STEP 1: THE SHARED SETUP (Everyone runs this first)

```
library(magick)
```

```
Linking to ImageMagick 6.9.13.29
Enabled features: cairo, fontconfig, freetype, heic, lcms,
pango, raw, rsvg, webp
Disabled features: fftw, ghostscript, x11
```

```
library(image.libfacedetection)

# Set Working Directory
setwd("~/Documents/Spring 2026 Courses/DS for Public Policy/face_

# Function to extract highest AI confidence
get_max_confidences <- function(folder_path, sample_size = 80) {
  image_files <- list.files(folder_path, full.names = TRUE, patte
  image_files <- image_files[!is.na(image_files)]
  if(length(image_files) == 0) return(c())

  actual_sample <- min(sample_size, length(image_files))
  sampled_files <- sample(image_files, actual_sample)
  max_confs <- numeric(actual_sample)

  for (i in 1:actual_sample) {
    img <- tryCatch(image_read(sampled_files[i]), error = functic
    if (!is.null(img)) {
      faces <- image_detect_faces(img)
      if (length(faces$detections$confidence) > 0) {
        max_confs[i] <- max(faces$detections$confidence, na.rm =
      } else {
        max_confs[i] <- 0
      }
    }
```

```
  }
  return(max_confs)
}

# Function to calculate Detection Rate and 95% Confidence Interva
calc_stats <- function(conf_vector, threshold) {
  n <- length(conf_vector)
  successes <- sum(conf_vector >= threshold)
  p <- successes / n
  margin_error <- 1.96 * sqrt((p * (1 - p)) / n)
  lower_bound <- max(0, (p - margin_error) * 100)
  upper_bound <- min(100, (p + margin_error) * 100)
  return(list(rate = p * 100, lower = lower_bound, upper = upper_
}

# The thresholds we want to test
thresholds <- c(50, 60, 70, 80, 90)
```

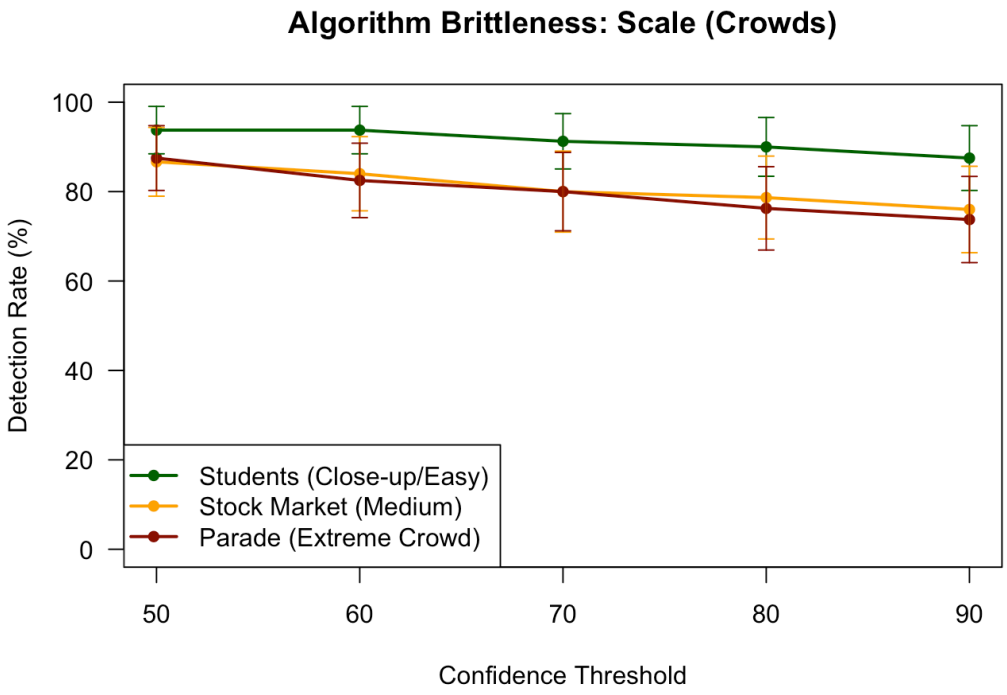
STEP 2: SCALE (Crowds & Distance)

```
print("Scanning Scale Categories...")
```

```
[1] "Scanning Scale Categories..."
```

```
data_parade <- get_max_confidences("WIDER_train/images//0--Para
data_stock  <- get_max_confidences("WIDER_train/images//15--Sto
data_student <- get_max_confidences("WIDER_train/images//29--St

# Calculate Stats
rate_parade <- sapply(thresholds, function(t) calc_stats(data_pa
low_parade <- sapply(thresholds, function(t) calc_stats(data_pa
up_parade <- sapply(thresholds, function(t) calc_stats(data_para

rate_stock <- sapply(thresholds, function(t) calc_stats(data_st
low_stock <- sapply(thresholds, function(t) calc_stats(data_sto
up_stock <- sapply(thresholds, function(t) calc_stats(data_stoc

rate_student <- sapply(thresholds, function(t) calc_stats(data_
low_student <- sapply(thresholds, function(t) calc_stats(data_s
up_student <- sapply(thresholds, function(t) calc_stats(data_st

# Plot
plot(thresholds, rate_student, type="o", col="darkgreen", pch=1
     main="Algorithm Brittleness: Scale (Crowds)", xlab="Confid
lines(thresholds, rate_stock, type="o", col="orange", pch=16, l
lines(thresholds, rate_parade, type="o", col="darkred", pch=16,
```

```
arrows(thresholds, low_student, thresholds, up_student, length=(
arrows(thresholds, low_stock, thresholds, up_stock, length=0.05,
arrows(thresholds, low_parade, thresholds, up_parade, length=0.(

legend("bottomleft", legend=c("Students (Close-up/Easy)", "Stocl
       col=c("darkgreen", "orange", "darkred"), lty=1, pch=16, 
```



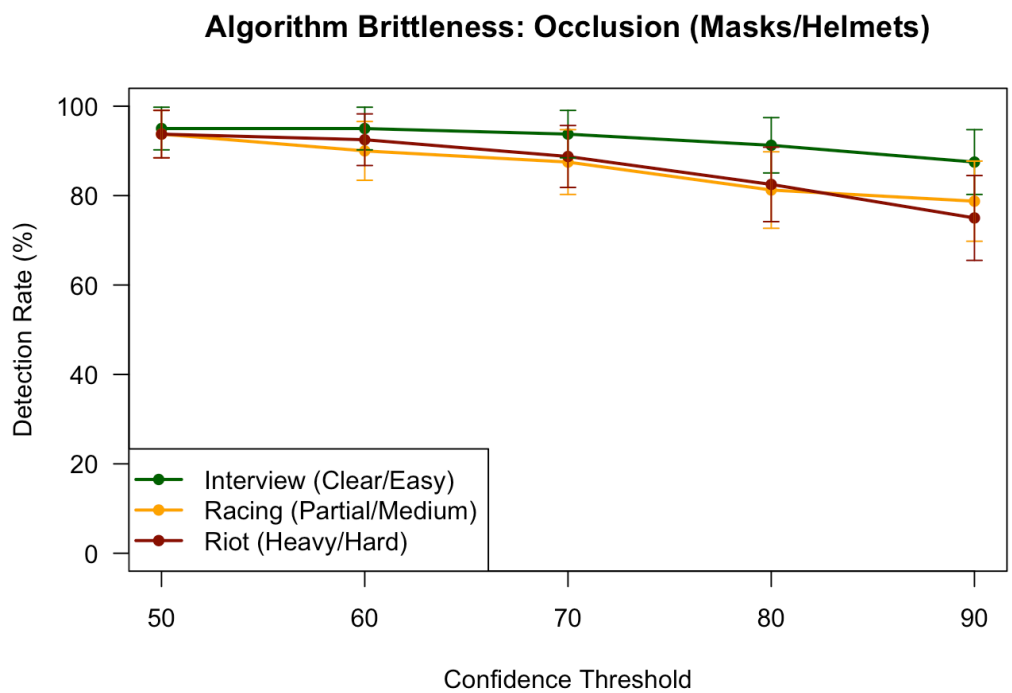
STEP 3: OCCLUSION (Obstructions & Masks)

```
print("Scanning Occlusion Categories...")
```

```
[1] "Scanning Occlusion Categories..."
```

```
data_riot <- get_max_confidences("WIDER_train/images//3--Riot")
data_racing <- get_max_confidences("WIDER_train/images//42--Car_
data_interview <- get_max_confidences("WIDER_train/images//13--

# Calculate Stats
rate_riot <- sapply(thresholds, function(t) calc_stats(data_riot
low_riot <- sapply(thresholds, function(t) calc_stats(data_riot,
up_riot <- sapply(thresholds, function(t) calc_stats(data_riot,

rate_racing <- sapply(thresholds, function(t) calc_stats(data_ra
low_racing <- sapply(thresholds, function(t) calc_stats(data_rac
up_racing <- sapply(thresholds, function(t) calc_stats(data_rac

rate_interview <- sapply(thresholds, function(t) calc_stats(data
```

```
low_interview <- sapply(thresholds, function(t) calc_stats(data_
up_interview <- sapply(thresholds, function(t) calc_stats(data_

# Plot
plot(thresholds, rate_interview, type="o", col="darkgreen", pch=
     main="Algorithm Brittleness: Occlusion (Masks/Helmets)", x
lines(thresholds, rate_racing, type="o", col="orange", pch=16,
lines(thresholds, rate_riot, type="o", col="darkred", pch=16, lv

arrows(thresholds, low_interview, thresholds, up_interview, leng
arrows(thresholds, low_racing, thresholds, up_racing, length=0.
arrows(thresholds, low_riot, thresholds, up_riot, length=0.05, a

legend("bottomleft", legend=c("Interview (Clear/Easy)", "Racing
       col=c("darkgreen", "orange", "darkred"), lty=1, pch=16,
```



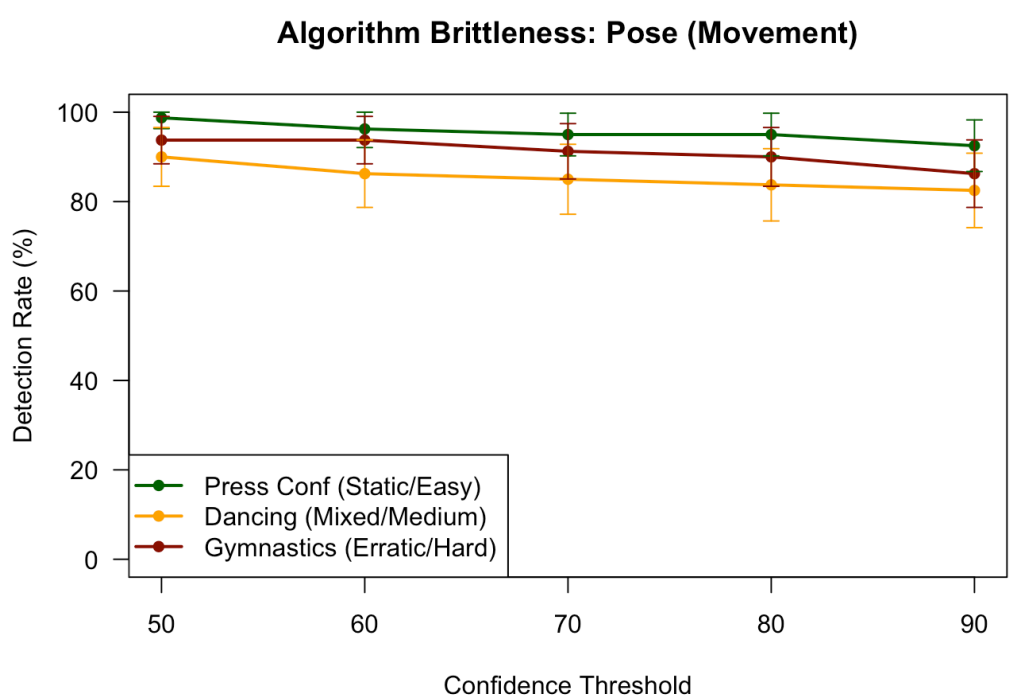
STEP 4: POSE (Movement & Angles)

```
print("Scanning Pose Categories...")
```

```
[1] "Scanning Pose Categories..."
```

```
data_gym <- get_max_confidences("WIDER_train/images//40--Gymnas
data_dance <- get_max_confidences("WIDER_train/images//4--Dancin
data_press <- get_max_confidences("WIDER_train/images//9--Press_

# Calculate Stats
```

```
rate_gym <- sapply(thresholds, function(t) calc_stats(data_gym,
low_gym <- sapply(thresholds, function(t) calc_stats(data_gym, 
up_gym <- sapply(thresholds, function(t) calc_stats(data_gym, t

rate_dance <- sapply(thresholds, function(t) calc_stats(data_dar
low_dance <- sapply(thresholds, function(t) calc_stats(data_dan
up_dance <- sapply(thresholds, function(t) calc_stats(data_dance

rate_press <- sapply(thresholds, function(t) calc_stats(data_pre
low_press <- sapply(thresholds, function(t) calc_stats(data_pres
up_press <- sapply(thresholds, function(t) calc_stats(data_press

# Plot
plot(thresholds, rate_press, type="o", col="darkgreen", pch=16,
     main="Algorithm Brittleness: Pose (Movement)", xlab="Confic
lines(thresholds, rate_dance, type="o", col="orange", pch=16, lw
lines(thresholds, rate_gym, type="o", col="darkred", pch=16, lwc

arrows(thresholds, low_press, thresholds, up_press, length=0.05,
arrows(thresholds, low_dance, thresholds, up_dance, length=0.05,
arrows(thresholds, low_gym, thresholds, up_gym, length=0.05, ang

legend("bottomleft", legend=c("Press Conf (Static/Easy)", "Danc
       col=c("darkgreen", "orange", "darkred"), lty=1, pch=16, 
```




```
# ==========================================
```

```r
# WIDER FACE COMPREHENSIVE SUBSET ANALYSIS
# Subsets based on Yang et al. (2016) parameters
# ==========================================

library(magick)
library(image.libfacedetection)

# 1. Define the Groupings based on the 29 Categories
# (b) SCALE SUBSETS
scale_large  <- c("1--Handshaking", "16--Award_Ceremony", "20--
scale_medium <- c("12--Group", "14--Traffic", "22--Picnic", "56-
scale_small  <- c("0--Parade", "3--Riot", "28--Sports_Fan", "61-

# (c) OCCLUSION SUBSETS
occ_none    <- c("1--Handshaking", "16--Award_Ceremony", "20--Fa
occ_partial <- c("4--Dancing", "5--Car_Accident", "34--Baseball'
occ_heavy   <- c("3--Riot", "9--Press_Conference", "39--Ice_Fish

# (d) POSE SUBSETS
pose_typical  <- c("16--Award_Ceremony", "1--Handshaking", "56--
pose_atypical <- c("30--Gymnastics", "40--Yoga", "4--Dancing", "

# (a) OVERALL SUBSETS (Averaging the extremes)
overall_easy   <- unique(c(scale_large, occ_none, pose_typical))
overall_medium <- unique(c(scale_medium, occ_partial))
overall_hard   <- unique(c(scale_small, occ_heavy, pose_atypical

# 2. Helper function to calculate aggregate rates for a subset o
get_subset_rates <- function(folder_list, thresholds = c(50, 60,
  all_confs <- c()
  for (folder in folder_list) {
    path <- paste0("WIDER_train/images//", folder)
    confs <- get_max_confidences(path, sample_size) # Uses your
    all_confs <- c(all_confs, confs)
  }

  rates <- sapply(thresholds, function(t) {
    success <- sum(all_confs >= t, na.rm = TRUE)
    return((success / length(all_confs)) * 100)
  })
  return(rates)
}

# 3. Calculate all rates
thresholds <- c(50, 60, 70, 80, 90)

print("Processing Scale Subsets...")
```

```
[1] "Processing Scale Subsets..."
```

```r
rate_b1_large <- get_subset_rates(scale_large)
rate_b2_med   <- get_subset_rates(scale_medium)
rate_b3_small <- get_subset_rates(scale_small)

print("Processing Occlusion Subsets...")
```

```
[1] "Processing Occlusion Subsets..."
```

```r
rate_c1_none <- get_subset_rates(occ_none)
rate_c2_part <- get_subset_rates(occ_partial)
rate_c3_heav <- get_subset_rates(occ_heavy)

print("Processing Pose Subsets...")
```

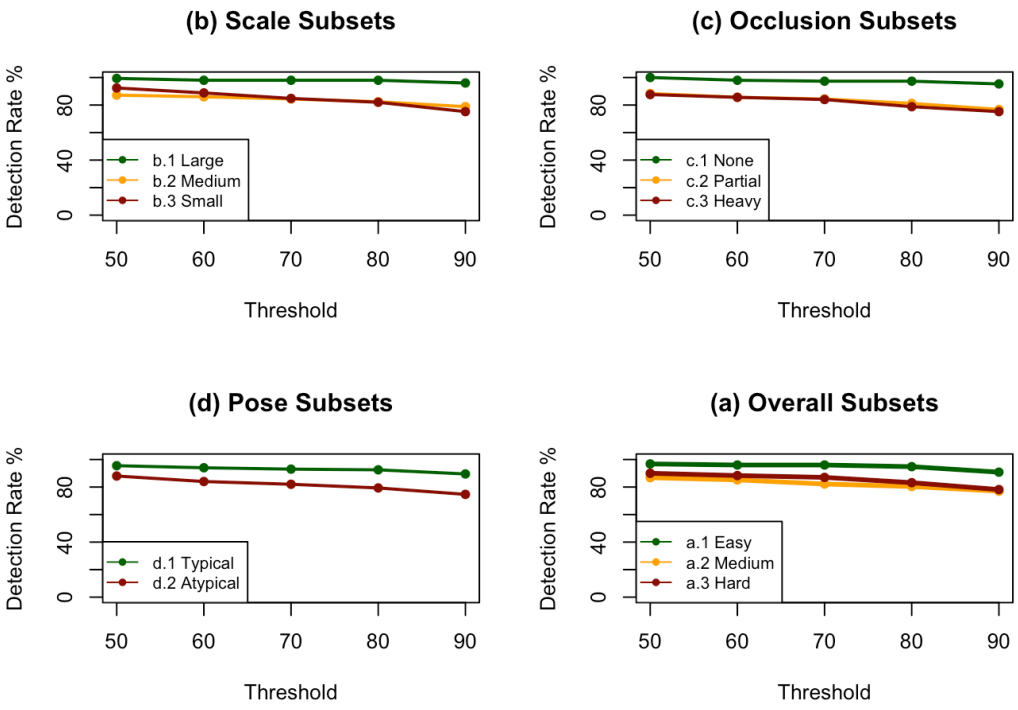
```
[1] "Processing Pose Subsets..."
```

```r
rate_d1_typ  <- get_subset_rates(pose_typical)
rate_d2_atyp <- get_subset_rates(pose_atypical)

print("Processing Overall Subsets...")
```

```
[1] "Processing Overall Subsets..."
```

```r
rate_a1_easy <- get_subset_rates(overall_easy)
rate_a2_med  <- get_subset_rates(overall_medium)
rate_a3_hard <- get_subset_rates(overall_hard)

# 4. Generate the Final Plots
par(mfrow=c(2,2)) # 2x2 grid for the 4 plots

# Plot B: Scale
plot(thresholds, rate_b1_large, type="o", col="darkgreen", pch=1
lines(thresholds, rate_b2_med, type="o", col="orange", pch=16, 
lines(thresholds, rate_b3_small, type="o", col="darkred", pch=16
legend("bottomleft", legend=c("b.1 Large", "b.2 Medium", "b.3 Sr

# Plot C: Occlusion
plot(thresholds, rate_c1_none, type="o", col="darkgreen", pch=16
lines(thresholds, rate_c2_part, type="o", col="orange", pch=16,
lines(thresholds, rate_c3_heav, type="o", col="darkred", pch=16,
legend("bottomleft", legend=c("c.1 None", "c.2 Partial", "c.3 He

# Plot D: Pose
plot(thresholds, rate_d1_typ, type="o", col="darkgreen", pch=16,
```

```
lines(thresholds, rate_d2_atyp, type="o", col="darkred", pch=16,
legend("bottomleft", legend=c("d.1 Typical", "d.2 Atypical"), c

# Plot A: Overall
plot(thresholds, rate_a1_easy, type="o", col="darkgreen", pch=1
lines(thresholds, rate_a2_med, type="o", col="orange", pch=16,
lines(thresholds, rate_a3_hard, type="o", col="darkred", pch=16,
legend("bottomleft", legend=c("a.1 Easy", "a.2 Medium", "a.3 Ha
```




```
# ==========================================
# WIDER FACE COMPREHENSIVE SUBSET ANALYSIS
# Subsets based on Yang et al. (2016) parameters
# ==========================================

library(magick)
library(image.libfacedetection)

# 1. Define the Groupings based on the 29 Categories
# (b) SCALE SUBSETS
scale_large  <- c("1--Handshaking", "16--Award_Ceremony", "20--
scale_medium <- c("12--Group", "14--Traffic", "22--Picnic", "56-
scale_small  <- c("0--Parade", "3--Riot", "28--Sports_Fan", "61-

# (c) OCCLUSION SUBSETS
occ_none    <- c("1--Handshaking", "16--Award_Ceremony", "20--Fa
occ_partial <- c("4--Dancing", "5--Car_Accident", "34--Baseball'
```

```
occ_heavy   <- c("3--Riot", "9--Press_Conference", "39--Ice_Fis

# (d) POSE SUBSETS
pose_typical  <- c("16--Award_Ceremony", "1--Handshaking", "56-
pose_atypical <- c("30--Gymnastics", "40--Yoga", "4--Dancing", "

# (a) OVERALL SUBSETS (Averaging the extremes)
overall_easy   <- unique(c(scale_large, occ_none, pose_typical)
overall_medium <- unique(c(scale_medium, occ_partial))
overall_hard   <- unique(c(scale_small, occ_heavy, pose_atypica

# 2. Helper function to calculate aggregate rates for a subset
get_subset_rates <- function(folder_list, thresholds = c(50, 60,
  all_confs <- c()
  for (folder in folder_list) {
    path <- paste0("WIDER_train/images//", folder)
    confs <- get_max_confidences(path, sample_size) # Uses your
    all_confs <- c(all_confs, confs)
  }

  rates <- sapply(thresholds, function(t) {
    success <- sum(all_confs >= t, na.rm = TRUE)
    return((success / length(all_confs)) * 100)
  })
  return(rates)
}

# 3. Calculate all rates
thresholds <- c(50, 60, 70, 80, 90)

print("Processing Scale Subsets...")
```

```
[1] "Processing Scale Subsets..."
```

```
rate_b1_large <- get_subset_rates(scale_large)
rate_b2_med   <- get_subset_rates(scale_medium)
rate_b3_small <- get_subset_rates(scale_small)

print("Processing Occlusion Subsets...")
```

```
[1] "Processing Occlusion Subsets..."
```

```
rate_c1_none <- get_subset_rates(occ_none)
rate_c2_part <- get_subset_rates(occ_partial)
rate_c3_heav <- get_subset_rates(occ_heavy)

print("Processing Pose Subsets...")
```

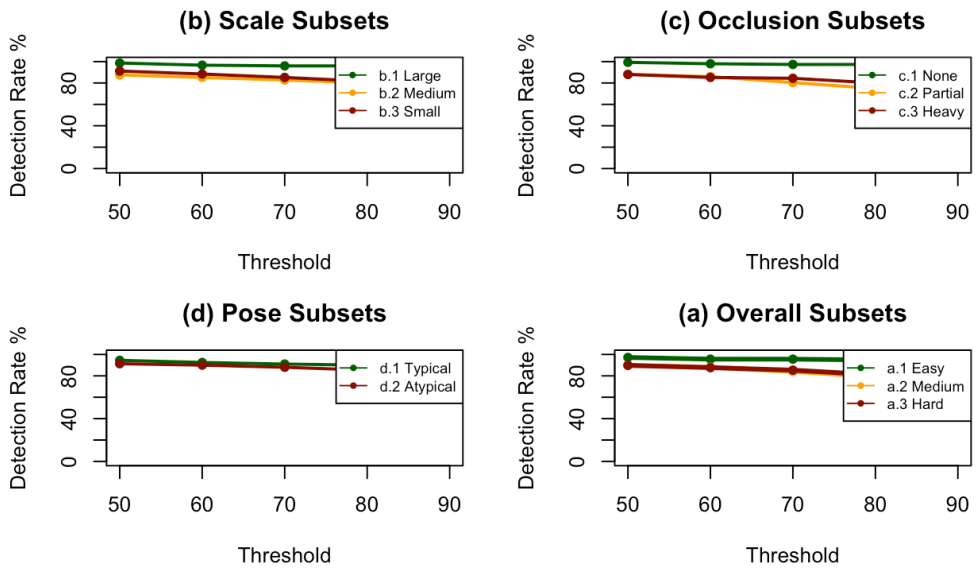
```
[1] "Processing Pose Subsets..."
```

```
rate_d1_typ  <- get_subset_rates(pose_typical)
rate_d2_atyp <- get_subset_rates(pose_atypical)

print("Processing Overall Subsets...")
```

```
[1] "Processing Overall Subsets..."
```

```
rate_a1_easy <- get_subset_rates(overall_easy)
rate_a2_med  <- get_subset_rates(overall_medium)
rate_a3_hard <- get_subset_rates(overall_hard)

# 4. Generate the Final Plots
# Reduced oma to 6 to restore plot visibility, mar adjusted for
par(mfrow=c(2,2), mar=c(4, 4, 3, 2), oma=c(6, 1, 1, 1))

# Plot B: Scale
plot(thresholds, rate_b1_large, type="o", col="darkgreen", pch=1
lines(thresholds, rate_b2_med, type="o", col="orange", pch=16, 
lines(thresholds, rate_b3_small, type="o", col="darkred", pch=16
legend("topright", legend=c("b.1 Large", "b.2 Medium", "b.3 Sma

# Plot C: Occlusion
plot(thresholds, rate_c1_none, type="o", col="darkgreen", pch=16
lines(thresholds, rate_c2_part, type="o", col="orange", pch=16,
lines(thresholds, rate_c3_heav, type="o", col="darkred", pch=16,
legend("topright", legend=c("c.1 None", "c.2 Partial", "c.3 Heav

# Plot D: Pose
plot(thresholds, rate_d1_typ, type="o", col="darkgreen", pch=16,
lines(thresholds, rate_d2_atyp, type="o", col="darkred", pch=16,
legend("topright", legend=c("d.1 Typical", "d.2 Atypical"), col=

# Plot A: Overall
plot(thresholds, rate_a1_easy, type="o", col="darkgreen", pch=16
lines(thresholds, rate_a2_med, type="o", col="orange", pch=16, 
lines(thresholds, rate_a3_hard, type="o", col="darkred", pch=16,
legend("topright", legend=c("a.1 Easy", "a.2 Medium", "a.3 Hard"

# Mapping Note - Using a slightly wider wrap and smaller text to
mapping_text <- "Category Mapping: (b) Scale: b.1 Large (Handsha

wrapped_lines <- strwrap(mapping_text, width = 130)

# Render centered text lines
```

```r
for (i in seq_along(wrapped_lines)) {
  mtext(wrapped_lines[i], side = 1, line = i + 1, outer = TRUE,
}
```



Category Mapping: (b) Scale: b.1 Large (Handshaking, Award Ceremony, Family Group), b.2 Medium (Group, Traffic, Picnic, Voter, People Marching), b.3 Small (Parade, Riot, Sports Fan, Street Battle, Demonstration). (c) Occlusion: c.1 None (Handshaking, Award Ceremony, Family Group), c.2 Partial (Dancing, Car Accident, Baseball, Tennis, Car Racing), c.3 Heavy (Riot, Press Conference, Ice Fishing, Raid, Rescue, Soldier Firing). (d) Pose: d.1 Typical (Award Ceremony, Handshaking, Voter, Celebration), d.2 Atypical